Breast Cancer Prediction Using Machine Learning Algorithms

Jayant Manem and Mahitha Bodicherla

Breast cancer is one of the most common cancers among women worldwide. Early detection plays a significant role in improving treatment outcomes and survival rates. With the growth of machine learning, data-driven models have become increasingly useful in helping healthcare professionals diagnose cancers more accurately and efficiently.

In this project, we used the Breast Cancer Wisconsin (Diagnostic) dataset from Kaggle. The goal was to build a machine learning model that can predict whether a tumor is benign or malignant, based on characteristics of the cell nuclei in a digital image of a fine needle aspirate (FNA). These features are numerical and represent measurements like radius, texture, and symmetry.

We used the Breast Cancer Wisconsin (Diagnostic) Data Set from Kaggle. This dataset has a class distribution of 357 benign values and 212 malignant values. There are many features that we took into consideration when using these machine learning algorithms. The dataset had columns such as:

ID: patient ID
Diagnosis: The diagnosis of breast tissues (M = malignant, B = benign)
Radius_mean: mean of distances from center to points on the perimeter
Texture_mean: standard deviation of gray-scale values
Perimeter_mean: mean size of the core tumor
Area_mean: mean size of the tumor
Smoothness_mean: mean of local variation in radius lengths
Compactness_mean: mean of perimeter^2 / area - 1.0
Concavity_mean: mean of severity of concave portions of the contour
Concave_points_mean: mean for number of concave portions of the contour
Symmetry_mean: mean symmetry between left and right breasts
Fractal_dimension_mean: mean for "coastline approximation" - 1

The dataset also consisted of the standard error of these features as well as the largest mean (“worst”) of these features. We decided to focus on the mean values of all these features in our model.

Before building any models, it's important to understand the structure and content of the dataset. EDA helps uncover patterns, relationships, and potential data quality issues. Here's a breakdown of the steps we took:

1. Data Structure and Summary

After loading the dataset, we first examined the shape and previewed the first few rows using .head() and .shape. This confirmed that there were originally 569 rows and 33 columns, including ID, diagnosis (target), and various statistical measures of tumor characteristics.

To simplify the analysis, we dropped columns that were either non-informative (id, Unnamed: 32) or redundant (all *_se and *_worst columns). This left us with 11 meaningful feature columns, all based on the mean values of different tumor characteristics.

```
# delete colums that are not needed
cancer_data = cancer_data.drop(['Unnamed: 32',
'id',"radius_se","texture_se",
"perimeter_se","area_se","smoothness_se","compactness_se","concavity_se","
concave points_se","symmetry_se",
"fractal_dimension_se","radius_worst","texture_worst","perimeter_worst","a
rea_worst","smoothness_worst","compactness_worst","concavity_worst","conca
ve points_worst","symmetry_worst",
"fractal_dimension_worst"], axis=1)
```

```
print("Shape:", cancer_data.shape)
print("Columns:", cancer_data.columns.tolist())
```

```
Shape: (569, 11)
Columns: ['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean']
```

Logistic Regression

We first created a logistic regression model using our cleaned data. We split the data into a training and testing group with a 30-70 split. We also scaled our data so that the dataset is more standardized.

```
[14] X_train, X_test, y_train, y_test = train_test_split(cancer_data.drop('diagnosis',axis=1),
                                                         cancer_data['diagnosis'], test_size=0.30)
```

```
[15] scaler = preprocessing.StandardScaler().fit(X_train)
```

```
[16] X_train_scaled = scaler.transform(X_train)
     X_test_scaled = scaler.transform(X_test)

     logmodel = LogisticRegression()
     logmodel.fit(X_train_scaled,y_train)
```

```
LogisticRegression
LogisticRegression()
```

The predictions for the model:

```
[17] predictions = logmodel.predict(X_test_scaled)
```

Metrics for logistic regression

```
print(classification_report(y_test,predictions))
print(confusion_matrix(y_test,predictions))
print(accuracy_score(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.92      0.99      0.96        97
           1       0.99      0.89      0.94        74

    accuracy                           0.95       171
   macro avg       0.95      0.94      0.95       171
weighted avg       0.95      0.95      0.95       171

[[96  1]
 [ 8 66]]
0.9473684210526315
```

As we can see from the result above, this model has an accuracy of 94.7%. This means that the model has correctly diagnosed 94.7% of the test values. We can see from the confusion matrix that 9 patients were diagnosed incorrectly by the model and that 162 were diagnosed correctly.

Random Forest

Next, we created a model using random forest. We split the data in the same way as we did for the logistic regression model.

```
#Random Forest

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_scaled, y_train)
rf_preds = rf_model.predict(X_test_scaled)

print(confusion_matrix(y_test, rf_preds))
print(accuracy_score(y_test, rf_preds))
print(classification_report(y_test, rf_preds))
```

```
[[92  5]
 [ 6 68]]
0.935672514619883
              precision    recall  f1-score   support

           0       0.94      0.95      0.94        97
           1       0.93      0.92      0.93        74

    accuracy                           0.94       171
   macro avg       0.94      0.93      0.93       171
weighted avg       0.94      0.94      0.94       171
```

Using the random forest model, we received an accuracy score of 93.5%. This is lower than the 94.7% that we got when we used logistic regression. From the confusion matrix, we can see that the model correctly diagnosed 160 patients and incorrectly diagnosed 11 patients.

Support Vector Machine

Lastly, we created a model using the support vector machine algorithm.

SVM (Support Vector Machine) Algorithm

```
[25] from sklearn.svm import SVC
```

```
model = SVC()
model.fit(X_train_scaled,y_train)
```

```
SVC
SVC()
```

```
[27] svm_predictions = model.predict(X_test_scaled)
```

Double-click (or enter) to edit

```
print(confusion_matrix(y_test,svm_predictions))
print("--------------------------------")
print(classification_report(y_test,svm_predictions))
print("--------------------------------")
print(accuracy_score(y_test, svm_predictions))
```

```
[[107   2]
 [  6  56]]
--------------------------------
              precision    recall  f1-score   support

           0       0.95      0.98      0.96       109
           1       0.97      0.90      0.93        62

    accuracy                           0.95       171
   macro avg       0.96      0.94      0.95       171
weighted avg       0.95      0.95      0.95       171

--------------------------------
0.9532163742690059
```

For this model, we received an accuracy score of 95.3% which is the highest accuracy among all three models. This model incorrectly diagnosed 9 people and correctly diagnosed 163 people.

After looking through the results of all three models, we concluded that the model created using the support vector machine algorithm was the best. It had the highest accuracy score and very good precision metrics.

Potential extensions of this project include exploring deep learning approaches (e.g., neural networks with TensorFlow or PyTorch) to capture more complex feature interactions, incorporating richer clinical and demographic data to improve predictive accuracy and real-world relevance, and deploying the model as a real-time web or API tool for integration into healthcare workflows. Additional enhancements could involve explainability techniques like SHAP or LIME for transparent predictions, as well as hyperparameter tuning and cross-validation to maximize performance and generalization.